NodeJS : Les bases

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Objectifs du cours

- ***Découvrir NodeJS à travers la création d'une API***
  - Qu'est-ce que NodeJS ?
  - Qu'est-ce que NPM ?
  - Qu'est-ce qu'une API ?
  - Comment créer et gérer mon API

I. Découvrir NodeJS

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Découvrir NodeJS

- ***Javascript*** est un langage de programmation utilisé pour interagir dans le cadre d'un navigateur
- ***NodeJS*** est une technologie nous permettant d'exécuter du ***Javascript*** côté ***serveur*** ou en tout cas dans ***un autre cadre*** que celui du navigateur
- ***NodeJS*** va aussi nous être utile pour faire tourner un ***serveur***, créer une ***API***, ou encore pour faire du ***task running***

Découvrir NodeJS

Pour installer NodeJS, allez sur le lien suivant et téléchargez le ***LTS***

https://nodejs.org/en/download

Découvrir NodeJS

Afin de vérifier si node est bien installé, vous pouvez lancer votre invité de commande et entrez la commande suivante :

```
PS C:\Users\Nicol> node --version
v18.17.0
```

Si une version apparaît c'est que tout est bon !

II. Utiliser NodeJS avec Express pour faire une API

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Utiliser NodeJS pour faire une API

- ***API*** (Application Programming Interface) est une manière de transmettre de la data entre plusieurs ***entités***. Par exemple, il existe une ***API*** Pokédex afin de récupérer l'ensemble des pokémons et des infos dessus : **https://pokeapi.co/**

- Grâce à votre ***API***, vous allez pouvoir stocker de la donnée mais aussi trigger des ***methods*** propre à l'***API***. Ces API peuvent être très puissantes comme ***DirectX*** qui est une ***API graphique*** très utilisée dans le jeu vidéo et qui sert à utiliser un code optimisé, performant et pré fait pour la réalisation de tâches complexes au niveau de la carte graphique. ***OpenGL*** en est un autre exemple

Utiliser NodeJS pour faire une API

- Nous allons, pour la suite du projet, utiliser ***NPM*** (Node Packet Manager)
- Cela va nous servir à installer des ***librairies***, faire tourner un serveur etc..
- ***NPM*** est un ***Task Runner***

Utiliser NodeJS pour faire une API

- Pour commencer, allez sur votre projet et ouvrez le terminal

```
npm init -y
```

Utiliser NodeJS pour faire une API

- Cela a dû vous créer un ***package.json*** :

```
{
  "name": "js-intro",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Utiliser NodeJS pour faire une API

- Maintenant nous pouvons commencer à installer ***express*** qui est un ***framework***

```
npm install express
```

- Express est alors disponible dans la liste des ***dependencies*** de notre projet. La liste se trouve dans la ***package.json***

- *Bon à savoir* : Au début de chaque projet, afin d'installer l'ensemble des modules, il est uniquement nécessaire de faire la commande suivante. NPM ira regarder dans le ***package.json*** et ira télécharger l'ensemble des ***dépendances*** nécessaire au projet

```
npm install
```

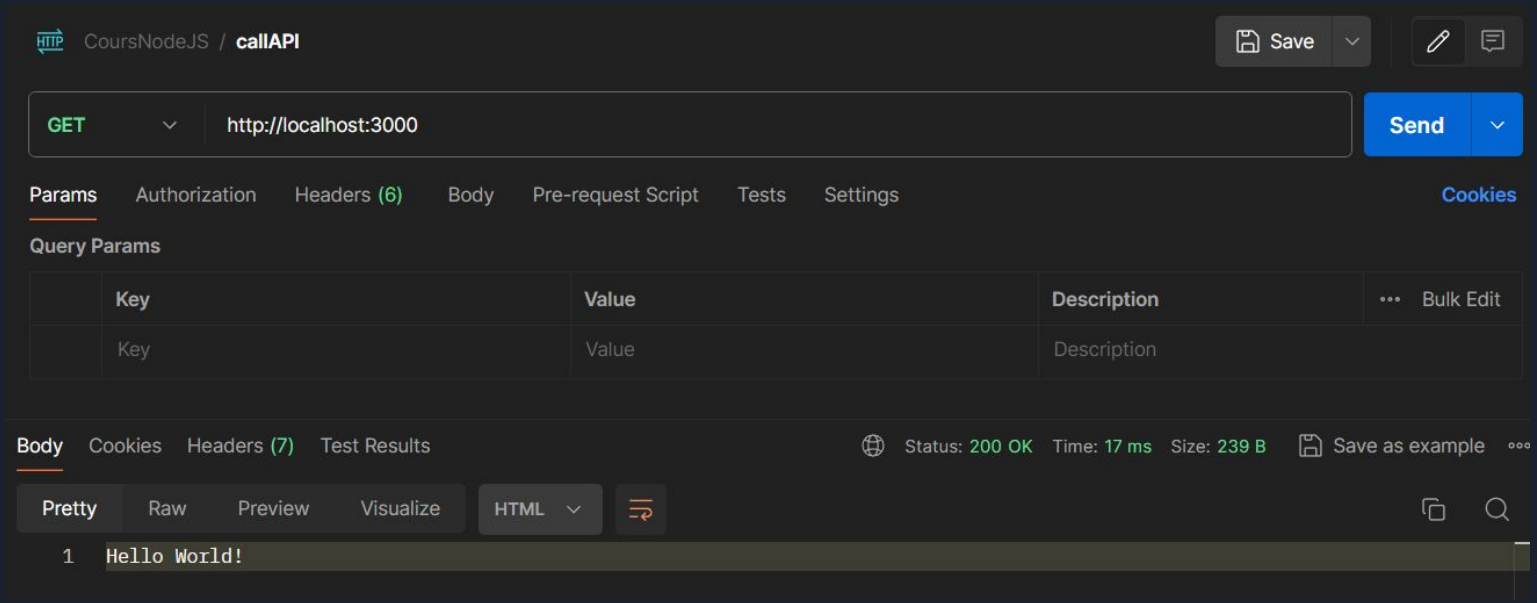
Utiliser NodeJS pour faire une API

- Faisons notre premières lignes ***NodeJS*** avec ***express***, utilisons le même exemple que dans la documentation : **https://expressjs.com/fr/**

```
1   const express = require('express')
2   const app = express()
3   const port = 3000
4
5   app.get('/', (req, res) => {
6       res.send('Hello World!')
7   })
8
9   app.listen(port, () => {
10      console.log(`Example app listening in 'http://localhost:${port}'`)
11  })
```

Utiliser NodeJS pour faire une API

- Pour faire des ***calls API***, vous pouvez utiliser ***Postman*** ou ***Insomnia***



Utiliser NodeJS pour faire une API

- Afin de contrôler l'entrée de nos ***request***, on va ajouter un ***middlewear*** qui va parser le ***body*** en ***JSON***

```
// Middlewear for the req
app.use(express.json())
```

- Maintenant, avec la documentation, essayez de faire votre premier ***POST*** qui renverra ce qui a été créé si la requête est un succès. Sinon, cela renverra un message d'erreur

Utiliser NodeJS pour faire une API

- Vous devriez avoir une requête qui ressemble à ça

```
const cats = [];
app.post('/cats/:id', (req, res) => {
    const id = req.params.id
    const { name, age } = req.body;

    if (!id || !name || !age) {
        return res.status(400).json({ message: "Please provide id, name, and age for the cat." });
    }

    const newCat = {
        id: id,
        data: {
            name: name,
            age: age
        }
    }
    cats.push(newCat);
    res.status(201).json(newCat);
})
```

Utiliser NodeJS pour faire une API

- Pour faire une requête ***GET***, on va utiliser le même tableau dans lequel nous avons stocker notre data

```
app.get('/cats/', (req, res) => {
    res.json(cats)
})

app.get('/cats/:id', (req, res) => {
    const catId = req.params.id;
    const cat = cats.find(cat => cat.id === catId);

    if (!cat) {
        return res.status(404).json({ message: "Cat not found." });
    }

    res.json(cat);
});
```

III. Sauvegarder notre data

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Utiliser NodeJS pour faire une API

- Maintenant que notre ***API*** fonctionne, il y a plusieurs moyens pour sauvegarder notre data :
  - On peut créer une ***BDD*** et gérer les call ***SQL*** via ***NodeJS***
  - On peut stocker notre data dans un ***JSON***
- Nous allons préférer la deuxième solution pour une question de ***simplicité***. Mais cette méthode ***n'est pas la plus pérenne***.
- Ce que nous allons faire c'est :
  - Si ***data.json*** n'existe pas, créer le fichier et le remplir à chaque ***method post***
  - Lire à chaque lancement du server le ***data.json***
  - Pour cela nous allons avoir besoin du ***module*** FS native à ***NodeJS***

Utiliser NodeJS pour faire une API

- ***FS*** est le module ***File System*** de ***NodeJS*** qui sert à gérer les fichiers système

```
// Manage the data.json creation & reading
const readDataFromFile = () => {
    try {
        const data = fs.readFileSync('data.json');
        cats = JSON.parse(data.cats);
    } catch (err) {
        cats = [];
    }
}

const writeDataToFile = () => {
    fs.writeFileSync('data.json', JSON.stringify({"cats": [...cats]}, null, 4));
}

if (!fs.existsSync('data.json')) {
    writeDataToFile();
} else {
    readDataFromFile();
}
```